

CI/CD with GitHub

Dave Wade-Stein

instructor.dws@gmail.com

Introduction to GitHub

Features, workflow, and where GitHub fits in modern software delivery

Overview of GitHub features

Starts w/source control, then adds collaboration, automation, security, and delivery tools around it

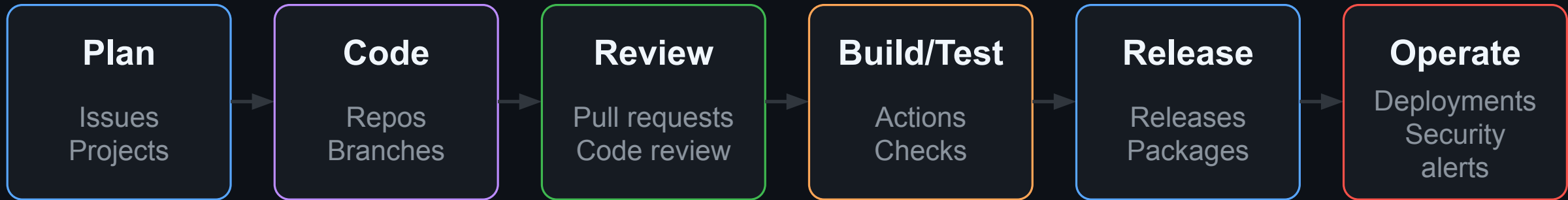
- **Code Hosting** - repositories, branches, commits, tags, history
- **Collaboration** - pull requests, review, comments, discussion, checks, merges
- **Project Tracking** - issues, labels, milestones, projects
- **Automation** - GitHub Actions workflows
- **Packages + Releases** - releases, artifacts, GitHub packages
- **Security** - Dependabot, code scanning, secret scanning
- **Documentation** - README files, Wikis, GitHub Pages

Key idea

GitHub can support the full software dev lifecycle from planning to deployment

How GitHub fits into the SDLC

GitHub can support the whole delivery path, but the repository stays at the center



Key idea

GitHub connects planning, code review, automation, releases, and security signals to the same source of truth: the repository

Comparison with other platforms

The core ideas are similar, but the ecosystem and defaults differ

GitHub

Developer ecosystem, open source visibility, Actions, security features, Copilot integration

GitLab

Broad integrated DevSecOps platform; strong built-in CI/CD identity

Bitbucket

Often chosen in Atlassian-centered teams; Jira and Confluence alignment

Azure DevOps

Strong fit for Microsoft/Azure-heavy organizations; Boards + Pipelines + Repos

Key idea

All 4 platforms provide Git hosting, code review, and integrated CI/CD. For teams already using GitHub, GitHub Actions provides a natural CI/CD choice without introducing another platform.

Introduction to CI/CD

Definition / Workflow / Best Practices

What is CI/CD?

- It's not just a tool or a YAML file, it's a development practice
- Specifically, a development practice which automates the path from code changes to tested, deliverable software
- **CI = Continuous Integration**
 - Developers merge code frequently
 - Automated checks run on each change
 - Problems are found early
- **CD = Continuous Delivery / Deployment**
 - what's the difference?

Continuous Delivery vs. Continuous Deployment

“CD” can mean two related but different things

Continuous Delivery

- Code is automatically built and tested
- A release candidate is prepared automatically
- A person usually signs off/approves the final release

Continuous Deployment

- Code is automatically built, tested, and deployed
- Changes where tests pass may go directly to production
- Requires strong automated testing and monitoring

The core collaboration loop

Most GitHub workflows revolve around a simple review loop: propose, discuss, verify, merge



The next change starts the loop again

CI/CD plugs into the “Checks” part of the loop—before the change is allowed to merge or deploy

Benefits of CI/CD

CI/CD helps teams make software changes w/more confidence + less manual effort

- Faster feedback
- Fewer manual release steps
- More consistent builds
- Earlier bug detection
- Better visibility into failures
- Safer, more repeatable deployments

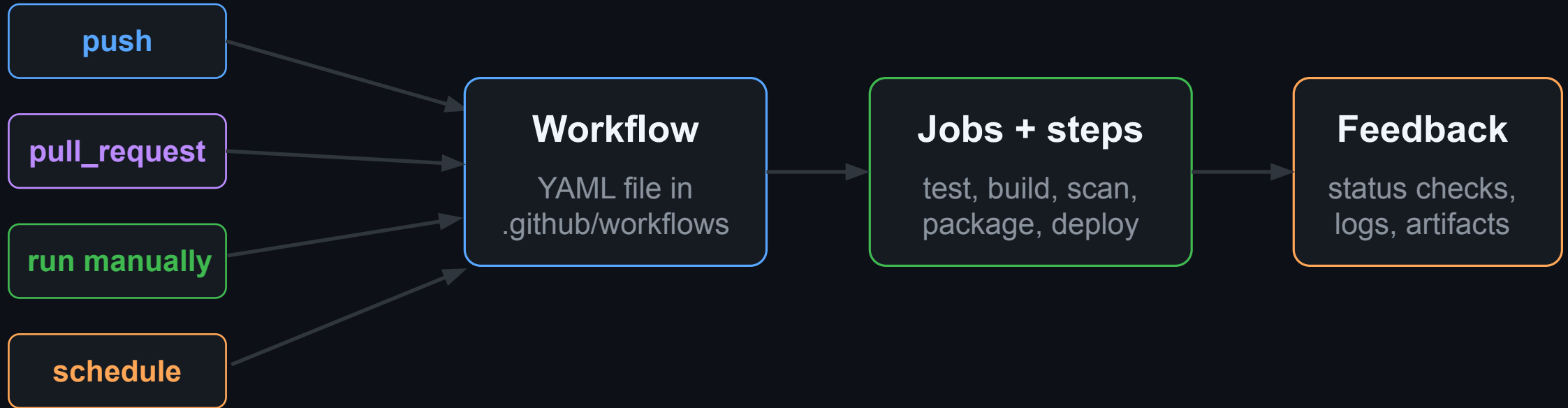
CI/CD Best Practices

A good pipeline should help the team move faster without hiding risk

- Run tests on pull requests
- Keep workflows fast and focused
- Fail early and clearly
- Protect secrets
- Use least-privilege permissions
- Separate build, test, and deploy concerns
- Make pipelines readable and maintainable

Where CI/CD appears in GitHub

CI/CD is not separate from GitHub collaboration, it is triggered by it



```
on:
  pull_request:
  push:
    branches: [main]
  ...
```

```
jobs:
  test:
    runs-on: ubuntu-latest
```

A typical workflow answers 4 questions

Think of a workflow as a small decision model before it is a YAML file

01 When should this run?

e.g., push, pull request, manual trigger, schedule

02 Where should this run?

e.g., GitHub-hosted runner or self-hosted runner

03 What work should be performed?

e.g., install dependencies, run tests, build an artifact, deploy

04 What should happen on failure?

e.g., stop the pipeline, block a pull request, prevent deployment

What to remember

These ideas will keep coming back throughout this material

GitHub ≠ Git

Git handles version control. GitHub wraps collaboration and automation around it

Pull requests are the hub

Review, discussion, checks, and merge decisions happen together

Actions connect code to automation

A workflow can test, build, scan, package, release, or deploy

Platform choice shapes workflow

Core concepts transfer, but syntax, integrations, and governance vary

Next: What CI/CD is and how a typical GitHub workflow moves from code to deployable software

Hands-On Exercise 1:

Your First GitHub Actions Workflow

In your GitHub Actions lab repository:

- Create a basic workflow
- Run it manually with `workflow_dispatch`
- Inspect the runner and step output
- Add `actions/checkout@v7` and observe what changes

What can trigger a workflow?

A workflow starts when GitHub sees an event that matches the rules in **on:**

```
on:  
  workflow_dispatch:  
  
  push:  
    branches: [main]  
    paths:  
      - src/**
```

Manual

`workflow_dispatch`

Run it from the
Actions tab

Push

`push`

Run when matching
commits land

Pull request

`pull_request`

Run when proposed
changes update

Filters

`branches / paths`

Limit which refs
and files matter

Hands-On Exercise 2:

Triggers and Filters

In your GitHub Actions lab repository:

- Add a push trigger alongside `workflow_dispatch`
- Use `github.event_name` to see what started the run
- Add branch and path filters
- Compare when the workflow does and does not run

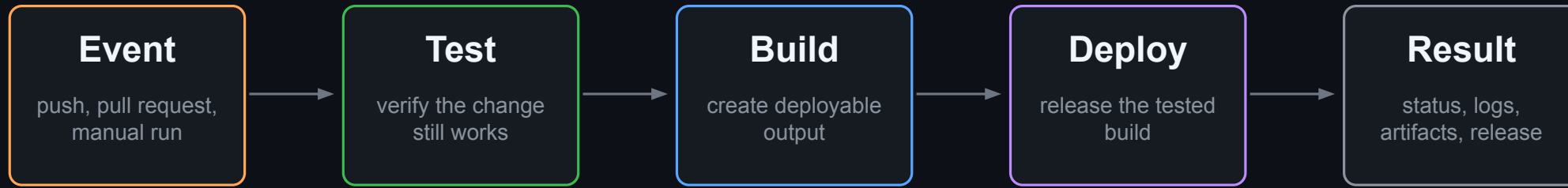
Defining a Typical GitHub Workflow with Actions

The familiar CI/CD progression: test, build, then deploy

Test **Build** **Deploy**

The mental model

A workflow is a recipe GitHub runs when something happens



The pipeline moves forward only when the earlier work succeeds

We use **needs**: to make one job wait for another, creating a Test → Build → Deploy pipeline

Where GitHub Actions files live

GitHub looks in specific repository folders for workflows and local actions

```
.github/  
├── workflows/  
│   ├── ci.yml  
│   └── deploy.yml  
└── actions/  
    └── setup-and-test/  
        └── action.yml
```

.github/workflows/

- Workflow YAML files live here
- GitHub discovers workflows from this folder

.github/actions/

- Common place for local custom actions
- Called with a relative **uses:** path

action.yml

- Metadata file that describes a custom action
- Defines inputs, outputs, and how the action runs

Workflow anatomy in YAML

Each job follows the same basic pattern: runner → steps → actions/commands

```
name: CI/CD

on:
  push:
    branches: [main]
  workflow_dispatch:

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm test

  build:
    needs: test
    ...

  deploy:
    needs: build
    ...
```

on = when should this workflow run?

jobs = what work should happen?

runs-on = where should a job run?

steps = what commands/actions run?

needs = what must finish first?

Test: prove the change still works

Test is the gate that asks whether this change broke anything we already depend on

What happens here?

- Check out the code
- Install dependencies
- Run automated tests
- Fail fast if something breaks

```
test:  
  runs-on: ubuntu-latest  
  steps:  
    - uses: actions/checkout@v4  
    - uses: actions/setup-node@v4  
    - run: npm ci  
    - run: npm test
```

Build: turn source into something deployable

The build usually depends on tests and creates an output other jobs can use

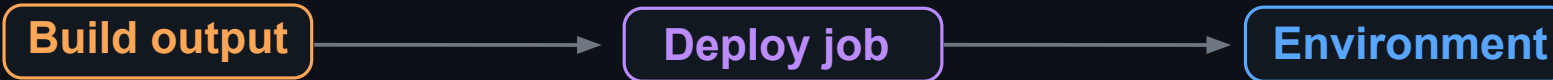


Common handoff build creates the output that will later be deployed

```
build:
  needs: test
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - run: npm ci
    - run: npm run build
    - uses: actions/upload-artifact@v4
  with:
    name: app
    path: dist/
```

Deploy: release the tested build

Deployment is where the pipeline touches a real target environment



```
deploy:
  needs: build
  runs-on: ubuntu-latest
  environment: production
  steps:
    - uses: actions/download-artifact@v5
      with:
        name: app
    - run: ./deploy.sh
```

Deployment concerns

- ◆ Which environment?
- ◆ Which secrets are needed?
- ◆ Is approval required?
- ◆ How do we roll back?

Here, **production** is the name of a GitHub Environment associated with this deployment job. That environment can hold secrets, approvals, and protection rules. We'll cover those later.

Putting the three jobs together

The **needs** keyword creates the basic pipeline: test → build → deploy

```
name: CI/CD

on:
  push:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm ci
      - run: npm test

  build:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm ci
      - run: npm run build
      - uses: actions/upload-artifact@v4
        with: ...

  deploy:
    needs: build
    environment: production
    steps:
      - uses: actions/download-artifact@v4
        with: ...
      - run: ./deploy.sh
```

1 Test

verify the code

2 Build

create deployable
output

3 Deploy

ship the build

Key insight

Job dependencies should match
the real delivery flow

Mini-lab: complete the pipeline

Fill in the blanks, then explain what would happen if test fails

```
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm test

  build:
    needs: ____
    runs-on: ubuntu-latest
    steps:
      - run: npm run build

  deploy:
    needs: ____
    environment: ____
    steps:
      - run: ./deploy.sh
```

1. What should build depend on?
2. What should deploy depend on?
3. What named environment would you associate with this deployment, and why?
4. What happens if npm test exits nonzero?

Hands-On Exercise 3:

Build a Multi-Job Pipeline

In your GitHub Actions lab repository:

- Add test, build, and deploy-demo jobs
- Connect them with needs
- Deliberately fail a job
- Observe what happens downstream

Running within your own infrastructure

Where GitHub Actions actually run and how to choose the right runner for the job

Hosted

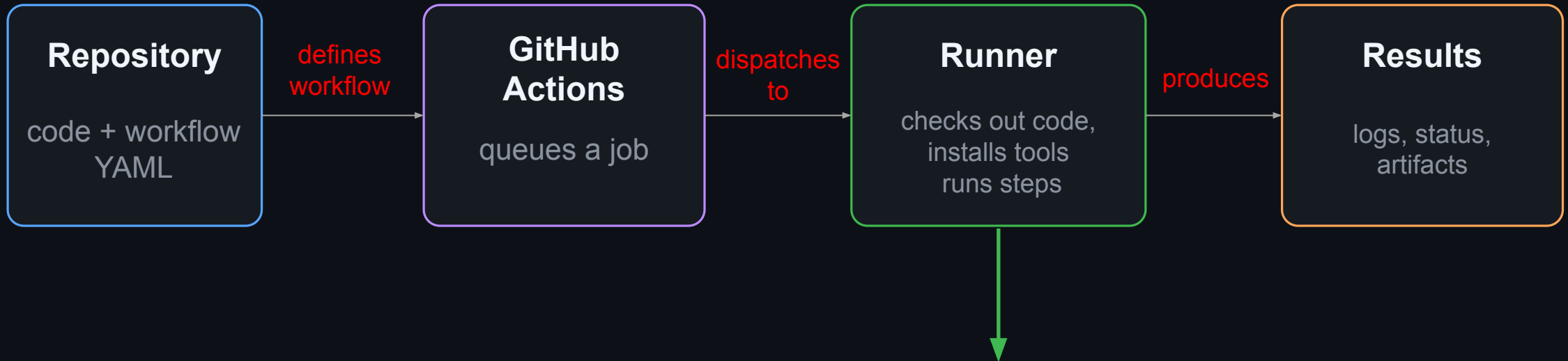
Self-hosted

Local

Advanced options

Intro to runners

A runner is the machine that executes a job in a GitHub Actions workflow



The runner is where your scripts actually run

That means OS, installed tools, network access, CPU/RAM, and security boundaries all matter

Hosted, self-hosted, and local runners

Same workflow concept, different execution environment

GitHub-hosted

- Managed by GitHub
- Clean VM / container per job
- Quickest default choice
- Great for public/cloud CI

Self-hosted

- Managed by your team
- Access to your network + tools
- Custom labels and groups
- More operational responsibility

Local

- Run/rehearse on a dev machine
- Useful for fast feedback
- Not identical to GitHub
- Best for debugging workflow logic

Self-hosted Runners

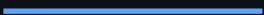
- machines you deploy and manage, then register with GitHub
- you might install the official GitHub Actions runner application on a server inside your company network and that would make it a self-hosted runner, which can be referred to in your YAML
- or if you're using something like **act** (runs GitHub Actions locally) on your Mac to test a workflow before pushing it, you're running it locally, but your Mac is not necessarily a GitHub runner (but it could be!)

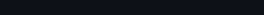
```
jobs:
  test:
    runs-on: ubuntu-latest

  deploy-internal:
    runs-on: [self-hosted, linux, prod-net]
```

When might you use your own runner?

Use the default hosted runners until you have a real reason not to

- 01**  **Private network access**

Deploy to internal systems, private package feeds, databases, or on-prem environments
- 02**  **Special hardware or OS needs**

GPU, ARM, licensed build tools, legacy runtime, unusual drivers
- 03**  **Control over the image**

Preinstalled dependencies, pinned toolchains, corporate hardening
- 04**  **Cost, speed, or capacity**

Reuse existing compute, reduce cold setup time, handle specialized workloads
- 05**  **Security and compliance boundaries**

Control data location, egress, audit approach, and isolation model

Self-hosted runners: the mental model

You bring the compute, GitHub sends it jobs

Install

runner app on a machine or VM

Register

repository, organization, or enterprise

Label / Group

control which jobs can target it

Run jobs

the runner polls GitHub and executes steps

Security notes

- Self-hosted runners can access your network and resources, so patch them, restrict access, and limit what workflows can run on them
- Also, be careful with untrusted code, public repos, secrets, and network access

Practical rule

Separate build/test runners from deployment runners unless there is a strong reason not to

Advanced runner options

As teams scale, runner choice becomes an operations and governance topic

Larger hosted runners

managed by GitHub, but with more control and resources

Runner groups

governance mechanism for letting orgs control access

Scale sets / ARC

k8s-based autoscaling for self-hosted runner fleets

Ephemeral runners

clean instance per job;
reduce residue between jobs

Custom images

preinstall your tools and dependencies for repeatable runs

Mini-lab: choose the right runner

For each scenario, choose a runner strategy and, if deployment is involved, identify target environment

```
# transitional YAML – discuss, then complete
jobs:
  deploy:
    runs-on: _____
    environment: _____
    steps:
      - uses: actions/checkout@v7
      - run: ./deploy.sh
```

**Runner choice: e.g.,
ubuntu-latest, [self-hosted,
linux, prod-net], local
rehearsal, runner group**

**Deployment Environment:
e.g., none, staging, prod,
etc.**

- 01 Every pull request**
Run unit tests on each proposed change
- 02 Private network deploy**
Deploy to a server reachable only inside your VPN
- 03 Licensed toolchain**
Build a native app that requires a licensed compiler
- 04 Local rehearsal**
Debug workflow syntax before pushing
- 05 Large organization**
Run hundreds of parallel jobs with runner groups or scale sets

Advanced Workflows

Making workflows more precise, reusable, extensible, and governable

Triggers

Composite actions

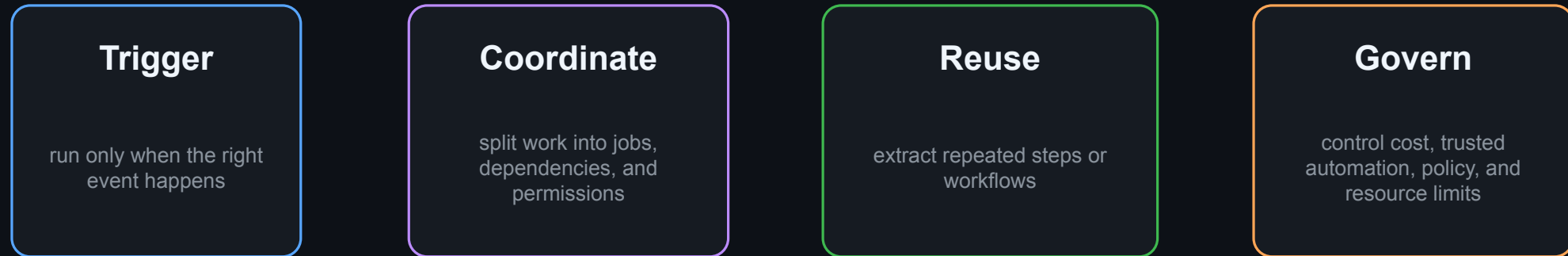
DRY reuse

Docker / JavaScript

Limits

From basic automation to production workflows

Advanced does not mean complicated, it means intentional



Production CI/CD is mostly about reducing surprise

The workflow should run when expected, reuse trusted building blocks, and fail in visible, understandable ways

Triggering workflows and actions

The **on:** block decides when a workflow run is created

```
name: ci

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
  workflow_dispatch:
  schedule:
    - cron: '0 6 * * 1'
```

Common event types

- **push** and **pull_request** are usual CI starting points
- **workflow_dispatch** gives you a manual button
- **schedule** runs at designated time via cron syntax

Important idea

An *event* triggers the *workflow run*; the workflow run executes the jobs and steps

Warning! Do not treat every workflow as “run on everything” (Triggers unnecessary work and makes important feedback harder to spot)

Make triggers specific

Filters keep workflows relevant, cheaper, and easier to understand

```
on:
  push:
    branches: [main]
    paths:
      - 'src/**'
      - 'package.json'

  pull_request:
    types: [opened,
synchronize, reopened]
    paths-ignore:
      - 'docs/**'
```

Branch filters

Only run for selected target branches or tags

Path filters

Skip work when irrelevant files change

Event activity types

Narrow events such as PR opened/synchronized/reopened

Path ignore filters

Do not run when only ignored paths changed

Warning! Combining branch and path filters means both must match!

Manual workflow runs

`workflow_dispatch` gives humans a controlled way to start a workflow on demand

```
on:
  workflow_dispatch:
    inputs:
      environment:
        description: 'Deploy environment'
        required: true
        type: choice
        options:
          - staging
          - production
      version:
        description: 'Version to deploy'
        required: false

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - run: ./deploy.sh
    env:
      TARGET_ENV: ${ inputs.environment }
```

Manual button

Adds a Run workflow button in the Actions UI

Inputs

Prompt the person running the workflow for values

Common uses

Deploy, release, rerun maintenance, generate reports

Important!

Manual does not mean uncontrolled; use environments, permissions, and reviews

Good for operations that should run when a person deliberately asks for them

DRY: Publishing and sharing repeated automation

There are two main reuse levels: reuse steps or reuse the workflow shape

Composite action

- Reusable group of steps
- Lives in a repo with `action.yml`
- Called from a step with `uses:`
- Good for “do this task”

Reusable workflow

- Reusable workflow file
- Triggered with `workflow_call:`
- Can define jobs, secrets, outputs
- Good for “run this pipeline”

Quick Distinction

Composite action = repeated steps

Reusable workflow = repeated CI/CD pattern

Composite actions

A composite action packages several steps into a reusable action that can be called with one **uses:** line

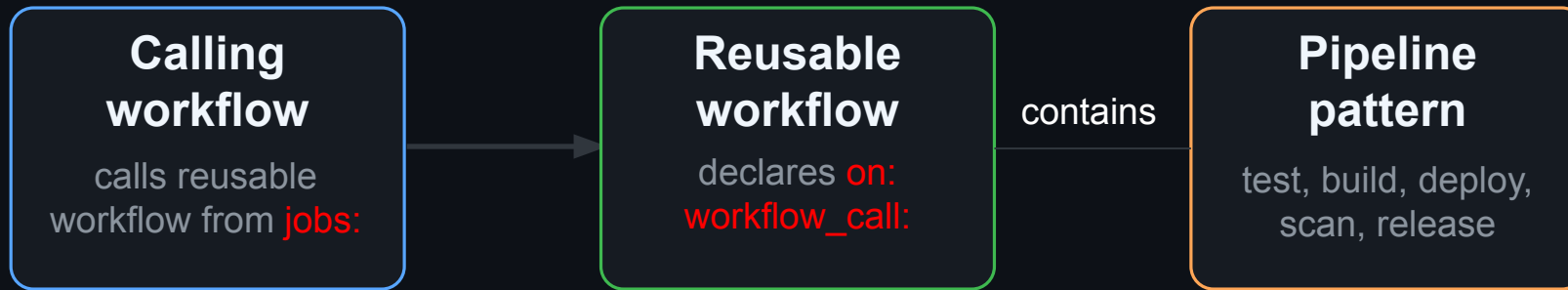


```
# workflow
steps:
  - uses: actions/checkout@v4
  - uses: ../.github/actions/setup-and-test
```

```
# action.yml
runs:
  using: 'composite'
  steps:
    - run: uv sync
      shell: bash
    - run: uv run pytest
      shell: bash
```

Reusable workflows

A reusable workflow packages one or more jobs, including their dependencies and overall pipeline structure so another workflow can call it with one job-level **uses:** line



```
# calling workflow
jobs:
  ci:
    uses: ../github/workflows/reusable-ci.yml
```

```
# reusable-ci.yml
name:
  workflow_call:

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - run: npm test
```

Leveraging Docker & JavaScript w/in Actions

Custom actions let you move beyond shell scripts

Composite

- Workflow-like steps and shell commands
- Simple packaging
- Good first custom action

Docker

- Bring your own runtime
- Consistent Linux environment
- Great for tools with dependencies

JavaScript

- Use Node.js APIs
- Fast startup
- Good for API calls and logic

```
# action.yml examples
runs: { using: 'composite', steps: [...] }
runs: { using: 'docker', image: 'Dockerfile' }
runs: { using: 'node20', main: 'dist/index.js' }
```

Managing usage restrictions and limits

As workflows scale up, we need guardrails so automation stays affordable, secure, and predictable

- 01 Minutes & storage**
GitHub-hosted runners consume billed/allocated minutes, and artifacts, logs, and caches consume storage
- 02 Concurrency**
Prevent duplicate or obsolete runs from piling up
- 03 Permissions**
Set least-privilege GITHUB_TOKEN permissions (grant only the read/write scopes each workflow or job actually needs)
- 04 Allowed actions**
Restrict which actions and reusable workflows can run
- 05 Branch/environment rules**
Protect releases with reviews, required checks, and environments

```
concurrency:  
  group: deploy-${{ github.ref }}  
  cancel-in-progress: true
```

```
permissions:  
  contents: read
```

Stop testing the old version of the code; test the newest version instead

Least privilege: workflow can only read the repository

Mini-lab: refactor an advanced workflow

Start from the transitional YAML, then make one refactoring decision in each category

```
name: release-candidate

on:
  push:
    branches: [main]

jobs:
  package:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: ./scripts/build.sh
      - run: ./scripts/scan.sh
      - run: ./scripts/package.sh
```

Trigger

Should this run on every push to main, only when relevant files change, or also manually?

Reuse

Is any part of this workflow repeated elsewhere? Should it become a composite action or reusable workflow?

Execution strategy

Does any task need Docker or JavaScript instead of ordinary workflow steps?

Guardrail

What permission or concurrency control would make this workflow safer?

What would you keep simple, and what is worth abstracting?

Environment variables and permissioning

Control configuration, protect credentials, and keep automation on the shortest safe leash

env

configuration values

secrets

sensitive values

permissions

allowed actions

policies

guardrails around risky
operations

Configure → Protect → Limit → Govern

Four levers for safer automation

Variables configure behavior, secrets provide sensitive access, permissions limit capability, and policies add guardrails

Variables

- Configuration values
- Environment names
- Build flags
- Shared constants

Secrets

- Tokens
- Passwords
- API keys
- Signing material

Permissions

- GITHUB_TOKEN scopes
- Read vs. write access
- Job-level exceptions

Policies

- Review gates
- Branch rules
- Environment protections
- Allowed actions

Good CI/CD design gives automation only the information, access, and authority it actually needs

Scoping environment variables

Use workflow, job, and step scopes intentionally

```
name: deploy-preview

on: [push]

env:
  NODE_VERSION: 22
  APP_ENV: preview

jobs:
  test:
    runs-on: ubuntu-latest
    env:
      TEST_REPORT: junit.xml
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: ${{ env.NODE_VERSION }}
      - run: npm test
        env:
          CI: true
```

Workflow env

Available to every job unless overridden

Job env

Available only inside one job

Step env

Most specific scope for one command or action

Shared configuration with vars

Repository-, organization-, or environment-level values reused across workflows

Keep configuration at the smallest scope that needs it

Variables are not secrets

Use variables for configuration; use secrets for values that would be harmful if exposed

```
# OK as configuration
env:
  REGION: us-east-2
  DEPLOY_TARGET: staging

# Use secrets for credentials
steps:
  - name: Publish package
    env:
      NPM_TOKEN: ${ secrets.NPM_TOKEN }
    run: npm publish
```

Variable

- Non-sensitive configuration
- Safe to expose
- Controls behavior

Secret

- Sensitive value
- Grants access or authority
- Should not appear in logs or source

Exposure test

Would it be safe if this value appeared in a workflow log? **If not, treat it as a secret**

Keeping secrets safe

Secrets should be stored securely, exposed narrowly, used carefully, and rotated deliberately

Store securely

- Repository, environment, or organization secrets
- External secret managers when needed

Expose narrowly

- Only to jobs or environments that need them
- Prefer protected environments

Use carefully

- Pass through env or action inputs
- Never print or echo secret values

Rotate and reduce

- Remove unused secrets
- Prefer short-lived credentials when possible

```
permissions:  
  id-token: write  
  contents: read
```

Prefer short-lived identity over long-lived cloud keys when possible

Masking helps protect logs, but it is not a complete security boundary

Limiting workflow permissions

Start with minimal permissions, then grant additional authority only to jobs that need it

```
name: build-and-release

on: [push]

permissions:
  contents: read

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm test

  release:
    # this job runs only when the expression
    # evaluates to true (see also exercise 5)
    if: startsWith(github.ref, 'refs/tags/')
    permissions:
      contents: write
      packages: write
    runs-on: ubuntu-latest
    steps:
      - run: ./scripts/release.sh
```

Default narrowly

Give the workflow minimal permissions by default

Elevate locally

Add broader permissions only to job that needs them

Separate risky work

Keep release authority out of ordinary test jobs

Gate execution

Use tags, branches, conditions, or environments intentionally

**Permissions control what a job can do;
conditions control when it can do it**

Policy gates for production work

A job may have permission to deploy and still be blocked until production policy is satisfied

```
jobs:
  deploy-prod:
    environment: production
    concurrency:
      group: prod-deploy
      cancel-in-progress: false
    if: startsWith(github.ref, 'refs/tags/')
    permissions:
      contents: read
      deployments: write
    steps:
      - run: ./deploy.sh
```

Environment protection

Put production behind a named GitHub environment

Required reviewers

Require approval before a deployment job proceeds

Branch or tag rules

Restrict which refs are allowed to deploy

Concurrency controls

Prevent overlapping production deployments

Production access should depend on both authorization and policy

Best practices checklist

Configure explicitly, grant minimally, and protect risky actions deliberately

- | | | |
|----|----------------------------------|---|
| 01 | Keep scope narrow | Define variables, secrets, and permissions only where they are needed |
| 02 | Separate sensitive work | Keep test and build jobs distinct from release or deployment authority |
| 03 | Prefer short-lived access | Use OIDC or temp credentials instead of long-lived secrets when practical |
| 04 | Make policy explicit | Use environments, reviewers, branch/tag rules, and conditions deliberately |
| 05 | Avoid accidental exposure | Do not echo secrets, over-share variables, or grant broad token permissions |

```
start with read only
permissions:
  contents: read
```

Putting it all together

Use the YAML feature that matches the question you are trying to answer

`env / vars`

What value does this command need?

`secrets`

How do we provide it safely?

`permissions`

What may this job do?

`triggers / if`

When does it run?

`environments /
policies`

What must be satisfied before risky work proceeds?

**The strongest workflows are explicit about
configuration, access, timing, and policy**

Mini-lab: harden a release workflow

Read the workflow, then identify configuration, secrets, permissions, and policy gates

```
name: Release

on:
  push:
    tags:
      - 'v*'

permissions:
  contents: read

env:
  APP_ENV: production

jobs:
  release:
    runs-on: ubuntu-latest
    environment: production
    permissions:
      contents: write
      packages: write
    steps:
      - uses: actions/checkout@v4
      - name: Publish
        env:
          API_TOKEN: ${ secrets.API_TOKEN }
        run: ./scripts/publish.sh
```

Configuration Which value is ordinary configuration?

Secret Which value is sensitive?

Permissions What is the default token posture?

Elevation Which job receives write access?

Trigger What causes the workflow to run?

Policy What can the production environment require?

What would change if **API_TOKEN** could be replaced with short-lived identity?

Artifacts & Dependencies

Preserving build output, sharing data between jobs, and managing storage intentionally

Create

Upload

Download

Deploy

Retain

```
build → upload artifact  
deploy → download same artifact
```

What are artifacts and what do they do?

Jobs run in clean environments, so outputs need a handoff mechanism

Files produced by one job that should survive beyond that job

Useful for downloads after a run and for passing files to downstream jobs

Test

- Reports
- Coverage
- Screenshots

Build

- Compiled app
- Static site
- Container metadata

Package

- Zip / tarball
- Installer
- Release bundle

Deploy

- Known build output
- Same tested files
- Promote without rebuilding

Retain

- Debug logs
- Evidence
- Audit trail

Creating artifacts from actions

Use `upload-artifact` when a job produces files that need to survive

```
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Build app
        run: npm run build

      - name: Upload build output
        uses: actions/upload-artifact@v4
        with:
          name: web-dist
          path: dist/
          retention-days: 7
```

Name intentionally

Use a name that makes sense in logs and later jobs

Upload the right path

Upload generated output, not the entire repo

Set retention

Keep short-lived CI output only as long as it is useful

Expect immutability

Treat as a build result, not a mutable workspace

Artifacts should answer “what exactly did this job produce?”

Using artifacts in downstream jobs

Use needs for order, then download the artifact in the dependent job

```
jobs:
  build: # from last slide
    runs-on: ubuntu-latest
    steps:
      - run: npm run build
      - uses: actions/upload-artifact@v4
        with:
          name: web-dist
          path: dist/

  deploy:
    runs-on: ubuntu-latest
    needs: build
    steps:
      - uses: actions/download-artifact@v4
        with:
          name: web-dist
          path: ./dist

      - name: Deploy
        run: ./scripts/deploy.sh ./dist
```

build job

Produces dist/
Uploads web-dist

deploy job

Downloads web-dist
Deploys same output

needs: build

Control job order with needs

Pass files between jobs

Deploy what you already built

Artifacts, caches, and packages

These are related storage tools, but they solve different problems

Artifact

- preserve workflow output
- reports, bundles, logs
- pass to later jobs or download

Cache

- reusable data that speeds up later runs
- dependencies and other files that are expensive to recreate
- not shipped with the release

Package

- publish for reuse or release
- libraries, container images, release bundles
- versioned and consumed later

cf. Job Workspace

- temp files on the runner
- available during the job
- gone after runner cleanup

```
# cache example: save and restore npm's local download cache
- uses: actions/cache@v4
  with:
    path: ~/.npm
    key: npm-${ runner.os }-${ hashFiles('package-lock.json')
  }}
```

Key Takeaway

Use artifacts when you care about preserving or passing the output of a workflow run

Managing artifacts and storage

Treat artifacts as useful evidence, not a permanent dumping ground

```
- uses: actions/upload-artifact@v4
  with:
    name: test-report-${{ github.run_number }}
    path: reports/
    retention-days: 5
```

Avoid uploading secrets, raw dependency directories, or huge intermediate folders

Retention

Shorter retention for noisy CI output
Longer retention for release evidence

Naming

Include role, version, platform, or run number when helpful

Scope

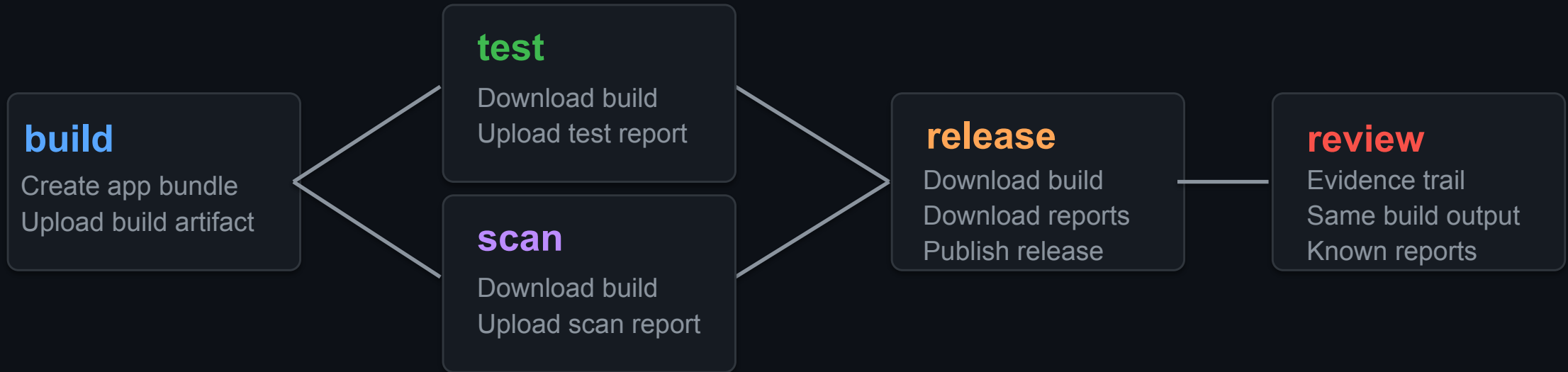
Upload only generated output and diagnostics

Cleanup

Use repository and organization settings to control retention

Designing artifact dependencies

Let the workflow graph show how outputs flow through the pipeline



Artifacts make data flow visible...The artifact you release is the same artifact you tested and scanned

Mini-lab: pass build output to deploy

Students add artifacts to a two-job workflow and confirm the handoff

```
jobs:
  build:
    steps:
      - run: mkdir dist && echo hello >
dist/index.html
      # TODO: upload dist as web-dist

  deploy:
    needs: build
    steps:
      # TODO: download web-dist
      - run: ls -R dist
```

Task 1

Upload dist/ from the build job

Task 2

Remove **needs: build**, predict what changes, then restore it

Task 3

Download the artifact into ./dist

Task 4

Verify that deploy sees file created by build

What would change with
multiple build targets?

Best practices checklist

Make artifacts intentional, traceable, and boring

- | | | |
|----------|--|--|
| 1 | Name artifacts by purpose | Make it obvious what the artifact contains |
| 2 | Keep retention short by default | Extend retention only for release evidence or audits |
| 3 | Build once, promote the same output | Avoid rebuilding differently in deploy jobs |
| 4 | Do not upload sensitive files | Treat artifacts as visible operational evidence |
| 5 | Separate cache from artifacts | Use cache for speed and artifacts for handoff |

```
name: web-dist-${{ github.run_number }}
path: dist/
retention-days: 7
```

Hands-On Exercise 4:

Pass Build Output Between Jobs

In your GitHub Actions lab repository:

- Upload build output as an artifact
- Download the artifact in a downstream job
- Verify that the exact build output moves between jobs
- Distinguish **needs** from artifact transfer

Configure and protect the workflow

After artifacts move output, decide which values and permissions the workflow should receive

```
env:  
  REGION: ${{ vars.DEPLOY_REGION }}  
  TOKEN: ${{ secrets.DEMO_TOKEN }}  
  
permissions:  
  contents: read
```

Variables

`vars`

Non-sensitive values
used to configure
workflow behavior

Secrets

`secrets`

Sensitive values
used to access
protected resources

Permissions

`permissions`

Limits what
GITHUB_TOKEN can
read/change

Least privilege

Give each job only
the access it needs
to do its work

Hands-On Exercise 5:

Variables, Secrets, and Permissions

In your GitHub Actions lab repository:

- Add a repository variable and a demo secret
- Use them safely in the workflow
- Add explicit `GITHUB_TOKEN` permissions
- Apply least privilege by limiting access to what each job needs

Hands-On Exercise 6:

Inputs, Concurrency, and Reusable Steps

In your GitHub Actions lab repository:

- Add a manual deployment input
- Add concurrency control
- Observe cancel-in-progress
- Move build preparation into
`.github/actions/prepare-build/action.yml`
- Call the composite action from the build job
- Run the workflow and inspect the results

Hands-On Exercise 7:

Cache vs. Artifact

In your GitHub Actions lab repository:

- Add `npm cache` before `npm ci`
- Run the workflow twice
- Keep uploading/downloading web-dist
- Explain the difference

Deployment Strategies

1. OK, so you've tested the code and built something you trust
2. The next question is—how do we get that version into a real environment?
3. That's what this section is about—how GitHub Actions connects the build side of CI/CD to actual deployment

Deployment Strategies

Push vs. pull, container images, and registries as the bridge from CI to production

Build

Turn source into a releasable unit

Publish

Store the image or package

Promote

Choose where it should run

Deploy

Update real infrastructure

Typical supporting pieces—not strict one-to-one pairs

These systems and controls often appear near each stage, but real pipelines can combine or shift responsibilities

GitHub Actions

Container Registry

Environment Gates

Runtime Platform

The deployment boundary

CI produces confidence, but deployment changes a running system

GitHub

Code, workflow,
checks, approvals

Release artifact

Build output, package,
container image

Infrastructure

Cluster, VM, PaaS,
edge, internal system

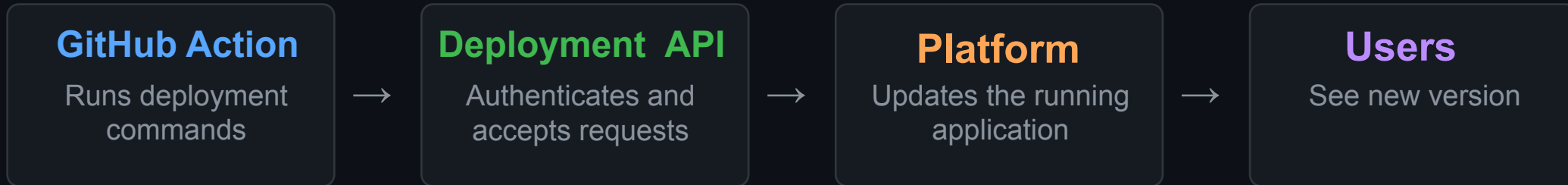
Trust boundary

- ◆ Who is allowed to initiate a deployment
- ◆ Where credentials live
- ◆ What can be rolled back
- ◆ How production changes are audited

Deployment is the boundary crossing between a trusted build and a real running system

Push-based deployment

GitHub Actions reaches out to the target environment and tells it what to deploy



Typical uses

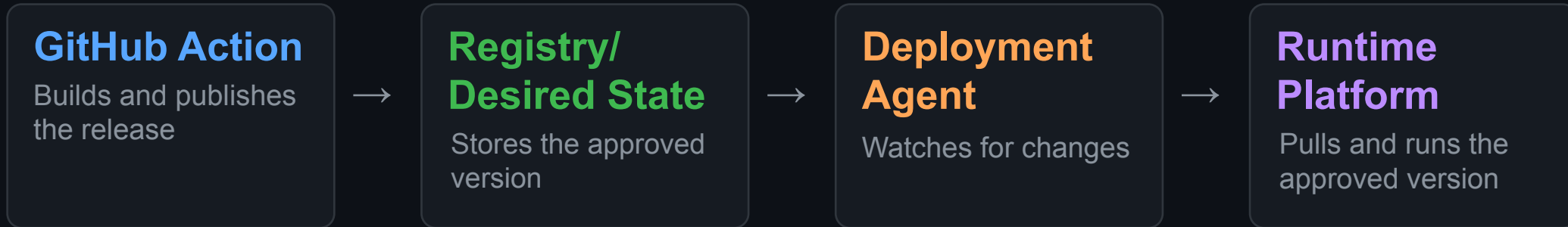
- PaaS deploys
- Static sites
- Serverless functions
- Internal systems GitHub can reach

Risks to Manage

- Deployment credentials must be protected
- Runner needs network access to the target
- Workflow has direct authority to change the environment

Pull-based deployment

GitHub publishes the desired version, and the target environment pulls it when appropriate



Why teams choose it

- The cluster does not expose deploy APIs to GitHub
- Credentials remain closer to the runtime environment
- The running system decides when it is safe to change

Common patterns

- GitOps controllers
- Kubernetes image pull and rollout
- Internal deployment agents
- Fleet or edge updates

Push vs. pull in one decision

Both can be correct, but they optimize for different constraints

Push

- Simpler to explain and implement
- Best when APIs are reachable
- Workflow owns the deploy command
- Easy fit for many cloud services

Pull

- Better for locked-down infrastructure
- Best when runtime owns reconciliation
- Workflow publishes desired state
- Natural fit for GitOps and clusters

Who should be allowed to make the final change to production?

Build, tag, and publishing container image

Turn the tested application into a versioned image that can be deployed later

```
name: publish-image

on:
  push:
    branches: [main]

jobs:
  image:
    runs-on: ubuntu-latest

    permissions:
      contents: read
      packages: write

    steps:
      - uses: actions/checkout@v4

      - uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${ github.actor }
          password: ${ secrets.GITHUB_TOKEN }

      - run: docker build -t ghcr.io/org/app:${ github.sha } .
      - run: docker push ghcr.io/org/app:${ github.sha }
```

What happens here

- Build the image from the tested source
- Tag it with a specific version or commit
- Authenticate to the registry
- Push the image to the registry

Why it matters

- Build once
- Store a known version
- Deploy that exact image later
- Make rollback and traceability easier

The container registry as a handoff point

The registry connects what CI builds with what the runtime deploys



What the registry provides

- Authentication and access
- Version discovery
- Retention and cleanup
- Provenance and audit trail

What deployment still decides

- Select the correct version
- Pull image from trusted source
- Roll out safely
- Rollback when needed

Promoting a known image through environments

Build once, identify the image clearly, then approve that same image for staging and production

01 sha-a1b2c3
exact build identity

02 staging
environment label

03 release-2026-08-02
named release

04 production
promotion target

A safer mental model

Build tag:
ghcr.io/org/app:\${{ github.sha }}

Environment tag:
ghcr.io/org/app:staging

Production deploy:
use image digest or approved release tag

Promotion rules

- Avoid relying on latest for production
- Prefer immutable tags or digests for auditability
- Promote a known image instead of rebuilding
- Make rollback a version choice, not a rebuild

Mini-lab: design/discuss the deployment

Choose how a known container image should move from the registry into a real environment

```
jobs:
  build_image:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        # TODO: login to registry
        # TODO: build and push image

  deploy_staging:
    needs: build_image
    runs-on: ubuntu-latest
    environment: staging
    steps:
      # TODO: choose push or pull strategy
      - run: ./scripts/deploy-staging.sh
```

Student tasks

- Add the minimum permissions
- Choose image tags intentionally
- Add an environment gate
- Explain rollback in one sentence

Best practices checklist

Make deployments repeatable, traceable, and reversible

01 Build once and promote the same image

02 Give each job only the permissions it needs

03 Use immutable tags or digests for traceability

04 Use stronger gates for higher-risk environments

05 Choose push or pull based on where deployment authority should live

06 Make rollback a version choice, not a rebuild

Hands-On Exercise 8:

Safe Staging Deployment

In your GitHub Actions lab repository:

- Rename `deploy-demo` to `deploy-staging`
- Add the `staging` GitHub Environment
- Add explicit read-only permissions
- Download the existing `web-dist` artifact
- Add ordinary deployment configuration
- Run the workflow and inspect the job dependency

Advanced Pipeline Actions

Move beyond test/build/deploy with security, compliance, and scheduled automation

**Security
scanning**

**License
edits**

**Scheduled
jobs**

**Quality
gates**

What belongs in an advanced pipeline

Advanced pipeline work usually manages risk, policy, and recurring tasks

Security scanning

Find code and dependency risks before they reach production

License audits

Block dependencies that conflict with company policy

Scheduled jobs

Run nightly, weekly, or maintenance automation

Quality gates

Turn findings into pass/fail decisions

The goal is not to make CI/CD fancy

The goal is not more automation it is useful automation that is visible and enforceable

Security scanning

Check code, dependencies, secrets, and images before they reach production

```
name: Security checks

on:
  pull_request:
  push:
    branches: [main]

jobs:
  scan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Run security tools
        run: echo "scanner goes here"
```

Code scan

Any vulnerabilities or risky coding practices? (e.g., CodeQL, SonarQube, Semgrep, Snyk Code)

Dependency scan

Any known vulnerable packages? (e.g., Dependabot, Snyk, Trivy, OWASP Dependency-Check)

Secret scan

Exposed API keys, tokens, credentials, private keys (e.g., GitHub Secret Scanning, Gitleaks, TruffleHog, GitGuardian)

Image scan

Vulnerabilities in container images and their included packages (e.g., Trivy, Grype, Snyk Container, Docker Scout)

Put results where developers already work

CodeQL as a starting point

Use GitHub's built-in code scanning integration before adding complexity

```
name: CodeQL

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
  schedule:
    - cron: '22 3 * * 1'

jobs:
  analyze:
    runs-on: ubuntu-latest
    permissions:
      # Give job only the access needed to read the
      # code and related resources + allow it to
      # write only security-analysis results
      security-events: write
      packages: read
      actions: read
      contents: read
    steps:
      - uses: actions/checkout@v4
      - uses: github/codeql-action/init@v3
      - uses: github/codeql-action/analyze@v3
```

When it runs

Run on PRs, pushes,
and a schedule

What it needs

Grant only what
the scan needs

Where results appear

Publish results to
code scanning alerts

How to refine

Start simple, then
adjust query suites
and builds

Start with the boring version first

License audits

A dependency can be technically safe and still be unacceptable

```
name: Dependency review

on: pull_request

jobs:
  dependency-review:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses:
        actions/dependency-review-action@v4
        with:
          fail-on-severity: high
          allow-licenses: MIT, Apache-2.0,
          BSD-3-Clause
```

License policy

Which licenses are
allowed or denied

Severity gate

Which vulnerabilities
should block the PR

Audit trail

Record the decision in
the pull request

**The CI system becomes a policy
enforcement point**

Scheduled jobs

Some work should run because time passed, not because code changed

```
name: Nightly maintenance

on:
  schedule:
    - cron: '30 6 * * 1-5'
  workflow_dispatch:

jobs:
  nightly:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Run recurring task
        run: ./scripts/nightly-check.sh
```

Nightly tests

run slow or comprehensive tests outside the PR path

Dependency Checks

Recheck dependencies and vulnerabilities as databases change

Cleanup jobs

Remove stale resources, previews, caches, or temporary data

Reports

Generate recurring reports even when the code has not changed

Use **workflow_dispatch**: so humans can run the same job on demand

Make advanced checks useful

More checks help only when results are meaningful and someone owns them

Noise

Tune checks so developers trust results (instead of ignoring them)

Cost

Run expensive check intentionally (rather than on every event)

Privilege

Give scanners only the permissions and secrets they need

Ownership

Who reviews findings and handles exceptions

Fail narrowly, report clearly, and make someone responsible for the result

Mini-lab: add advanced checks

Add useful checks without turning the workflow into a tool catalog

```
name: Advanced checks

on:
  pull_request:
  schedule:
    - cron: '0 7 * * 1'
  workflow_dispatch:

jobs:
  advanced-checks:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        # add dependency review
        # add a placeholder scanner
        # add a report artifact
```

1. Add dependency review with one policy decision
2. Add a scheduled and manual triggers
3. Preserve a scan summary as an artifact
4. Decide which findings should block a merge

What should fail immediately, and what should create a review task?

Putting It All Together

From workflow syntax to delivery thinking

From workflow syntax to delivery thinking

YAML describes the mechanism, but the workflow should express a delivery decision

```
on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm test
```

Syntax

keys, indentation,
events, jobs, steps



Intent

when to run, what to
verify, what to block



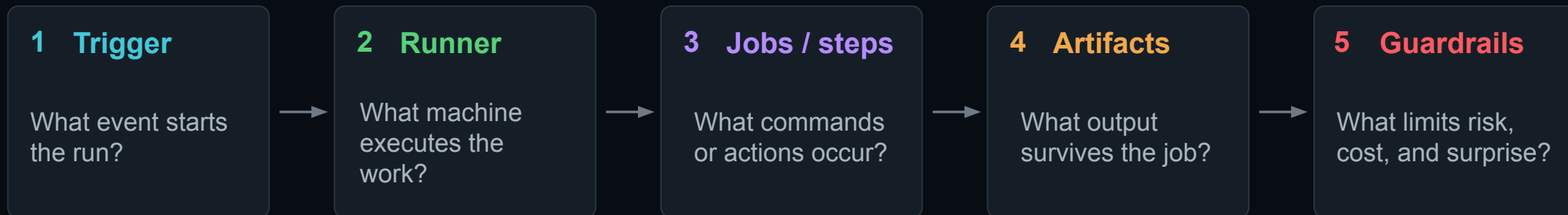
Outcome

fast feedback and
safer change

Good workflows answer why, not just how

The recurring pattern

Most workflows can be understood as a small number of connected decisions



A workflow is not just a script; it is a controlled delivery path

What good workflows have in common

The details vary, but strong workflows tend to share the same qualities

- 1 Specific triggers** Run when the right event or file change happens
- 2 Clear job boundaries** Separate test, build, scan, package, and deploy concerns
- 3 Reusable pieces** Extract repeated steps or repeated pipeline shape deliberately
- 4 Limited permissions** Grant only the access each workflow or job needs
- 5 Visible results** Expose status, logs, reports, artifacts, and approvals where people work

Good CI/CD reduces surprise for both developers and operators

Final mini-discussion

Use the course concepts to identify one practical improvement

What would you automate first?

A repeated check, build, report, release step, or maintenance task that currently depends on memory

What would you protect first?

A deployment, secret, package publish, or production action that needs clearer permissions or review

What would you simplify first?

A noisy, duplicated, over-triggered, or hard-to-debug workflow that creates avoidable friction

Pick one improvement and describe the trigger, runner jobs, outputs, and guardrails you would use

The best first automation is often small, visible, and easy to trust

Make important work repeatable, visible, and safer to change

That is the practical goal of CI/CD with GitHub Actions

Repeatable

The same checks and delivery steps run the same way each time

Visible

Results appear where teams review, decide, and improve

Safer

Risky changes have limits, evidence, and policy gates

CI/CD is not about making automation fancy **It's about making important work dependable**